Informatik Spektrum (2024) 47:153–162
https://doi.org/10.1007/s00287-024-01583-x

HAUPTBEITRAG

Grenzen verschiedener Betriebssysteme und Endgeräte überwinden durch neuartiges Desktop-as-a-Service

Der Desktop-as-a-Service DESIGN ermöglicht die Nutzung beliebiger Anwendungen in verschiedenen Einsatzszenarien aus dem Browserfenster heraus und löst damit die Grenzen bekannter Bereitstellungsstrategien auf

Alexander Atanassov[1] · Christian Baun[1] · Johannes Bouché[1]

Angenommen: 2. September 2024 / Online publiziert: 6. Dezember 2024
© The Author(s) 2024

Zusammenfassung
Die neuartige und freie Lösung DESIGN realisiert einen modernen Desktop-as-a-Service (DaaS), der den Betrieb unmodifizierter Linux/Windows-Anwendungen ermöglicht und durch den Verzicht auf eine spezielle Client-Software keine Änderungen am lokalen Client erfordert. Die Interaktion mit DESIGN und die Nutzung der integrierten Anwendungen geschehen vollständig über den Browser. Dieser Artikel stellt das moderne Architektur- und Nutzungskonzept des DaaS vor, beschreibt im Detail die Integration verschiedener Funktionen wie persistente Datenhaltung, Drucken oder Sound und erläutert deren charakteristische Performanceaspekte.

Einleitung

Die Auswahl der optimalen Bereitstellungsstrategie für IT-Anwendungen jeglicher Art ist eine alltägliche Herausforderung aller denkbaren Personengruppen in IT-Projekten, deren Beantwortung immer vom konkreten Anwendungsszenario und zahlreichen Nebenbedingungen abhängt. Konkret haben Nutzer, Administratoren, Projektentwickler etc. die Wahl zwischen

- verschiedenen Betriebssystemfamilien (Linux/UNIX und Windows),
- nativen Anwendungen oder SaaS („Web-Anwendungen“)
- sowie lokalem Betrieb („On-Premises“) oder Betrieb in einer Public Cloud inklusive verschiedener Hybrid-Cloud-Szenarien.

Die Vor- und Nachteile des traditionellen IT-Konzepts mit nativen, lokal bereitgestellten Anwendungen im Vergleich zu SaaS zeigen die Tab. 1 und 2. Der Betrieb nativer Anwendungen für die Betriebssystemfamilien Linux und Windows stellt bestimmte Anforderungen an das verwendete Endgerät hinsichtlich Hardware- und Software und verursacht einen hohen Aufwand für Administration bei gleichzeitigen Einschränkungen der Nutzbarkeit (u. a. in mobilen Szenarien). Software-as-a-Service bzw. Webanwendungen als Alternative bieten viele Vorteile hinsichtlich Nutzung und Administration, enthalten aber dafür andere Nachteile hinsichtlich der Nutzung von Legacy-Anwendungen.

Auch moderne VDI-Lösungen („Virtual-Desktop-Infrastruktur“) wie Azure Virtual Desktop oder Windows 365 sind nicht frei von Nachteilen, denn sie erhöhen die Komplexität (zusätzlicher Desktop), erfordern ein signifikantes Maß an Konfiguration und liegen in der Regel nicht als vollständig vertrauenswürdige, weil freie Software vor.

Die Motivation hinter dem Projekt DESIGN ist der Versuch, eine Bereitstellungsstrategie zu entwickeln, die die bekannten Vorteile (siehe Tab. 1) aufgreift, aber die Nachteile (siehe Tab. 2) nach Möglichkeit vermeidet.

✉ Christian Baun
christianbaun@fb2.fra-uas.de

[1] Frankfurt University of Applied Sciences,
Nibelungenplatz 1, 60318 Frankfurt am Main, Deutschland

Tab. 1 Vorteile lokal bereitgestellter nativer Anwendungen vs. SaaS

Lokal bereitgestellte Anwendungen („native Linux/Windows-Anwendungen“)	Software-as-a-Service – SaaS („Webanwendungen“)
Bestmöglicher Grad an Datenschutz und Datensicherheit (bei entsprechendem Administrationsaufwand)	Jeder Client mit modernem Browser ist prinzipiell zur Nutzung geeignet
Unterstützung von Legacy-Anwendungen	Synchronisation und Backups werden in der Regel automatisch vom Anbieter bereitgestellt (und sind je nach Anbieter mehr oder weniger vertrauenswürdig)
Vendor Lock-in ist ausgeschlossen	Kein Einrichtungsaufwand aus Nutzerperspektive (wenn es sich um eine Public Cloud handelt)

Tab. 2 Nachteile lokal bereitgestellter nativer Anwendungen vs. SaaS

Lokal bereitgestellte Anwendungen („native Linux/Windows-Anwendungen“)	Software-as-a-Service – SaaS („Webanwendungen“)
Client erfordert bestimmtes Betriebssystem oder eine Virtualisierungslösung	Hoher Einrichtungsaufwand aus Nutzerperspektive (wenn es sich um eine Private Cloud handelt)
Client muss vertrauenswürdig sein	Dienstanbieter muss vertrauenswürdig sein
Hoher Aufwand für die Administration	Risiko eines Vendor Lock-in
Fernzugriff ist möglich, erhöht aber den Administrationsaufwand zusätzlich	Netzwerkverbindung erforderlich
Üblicherweise keine automatische Synchronisation der Daten mit externen Ressourcen oder anderen Clients	Der Betrieb von Legacy-Anwendungen ist in den meisten Fällen nicht möglich
Üblicherweise keine automatische Datensicherung	Potenzielle Probleme bzgl. Datenschutz und Datensicherheit sind möglich

Web-Desktops als Inspiration für eine Verbesserung des Status Quo und die Entwicklung eines modernen DaaS

Ein prinzipiell interessantes Konzept zur Bereitstellung von Anwendungen bzw. von deren Funktionalität sind sogenannte Web-Desktops oder Webtops. Diese versuchen, auf Basis von Websprachen oder Webtechnologien wie HTML, JavaScript, Silverlight und PHP, Desktops im Webbrowser nachzubilden bzw. zu „emulieren“. In der Literatur werden solche Lösungen häufig zur Kategorie Desktop-as-a-Service hinzugezählt. Bekannte Lösungen für Web-Desktops sind eyeOS/oneye [8, 11], SilveOS [6], OS.js, CorneliOS oder daedalOS. Gemeinsamkeiten dieser Projekte sind, dass sie alle auf veralteten Technologien aufsetzen und ihre Weiterentwicklung zum aktuellen Zeitpunkt inaktiv ist. Der aus Nutzersicht am meisten ins Gewicht fallende Nachteil dürfte aber sein, dass keines dieser Projekte einen Zugriff auf bestehende Anwendungen für Linux oder Windows über einen Browser ermöglicht. Die Anforderung, dass alle Anwendungen als Webanwendungen neu implementiert werden müssten, war bei den Nutzern nicht durchsetzbar.

Dennoch können Lösungen wie eyeOS und das Nachfolgeprojekte oneye für neue Projekte und Konzepte inspirierend wirken, da sie zur Interaktion ausschließlich einen Browser voraussetzen, in Public Clouds sowie On-Premises lauffähig sind und ausschließlich freie Software verwenden, was einen Vendor Lock-in ausschließt.

Basierend auf den bereits in den Tab. 1 und 2 beschriebenen Vor- und Nachteilen verschiedener Bereitstellungsstrategien, der Motivation zur Verbesserung des Status Quo und inspiriert von der einfachen Bedienbarkeit der genannten Web-Desktops, entstand die DaaS-Lösung DESIGN. Die angestrebten funktionalen und nicht-funktionalen Anforderungen des neuartigen DaaS waren:

1. Die Möglichkeit der Integration unmodifizierter Linux/Windows-Anwendungen.
2. Der DaaS sollte nach Möglichkeit ausschließlich freie Software (Open Source) verwenden und somit auch langfristig einen Betrieb in verschiedenen Bereitstellungsstrategien ohne Vendor Lock-in ermöglichen.
3. Möglichkeit, den Fokus des DaaS-Lösung auf die integrierten Anwendungen zu lenken, und nicht auf einen oder mehrere weitere Desktops, die die Komplexität der täglichen Arbeit weiter vergrößern, da ein lokaler Desktop nicht komplett ersetzt werden kann.
4. Die Interaktion soll ausschließlich über den Browser erfolgen. Die Installation weiterer Client-Software wird somit vermieden und das Client-System bzw. das Host-Betriebssystem auf der Benutzerseite bleiben unverändert.
5. Der Betrieb des DaaS vor Ort und eine Nutzung als Service sollen gleichermaßen möglich sein. Diese Anforderung ist besonders in Szenarien relevant, die ein hohes Maß an Datenschutz voraussetzen.
6. Um eine möglichst hohe Zuverlässigkeit des Dienstes und ein hohes Maß an Datensicherheit zu erreichen, muss ein Clusterbetrieb möglich sein.

Anforderungen an die Architektur des DaaS und deren Implementierung

Die beschriebenen Eigenschaften des Gesamtsystems bestimmen die Anforderungen an die zu integrierenden Komponenten. Abb. 1 zeigt die Architektur der neuartigen DaaS-Lösung DESIGN.

Zur Integration der Linux- und Windows-Anwendungen (siehe Anforderung 1) sind Container und virtuelle Maschinen hilfreich. Sie verbessern durch den isolierten Betrieb die Sicherheit des Gesamtsystems. In der gewähl-

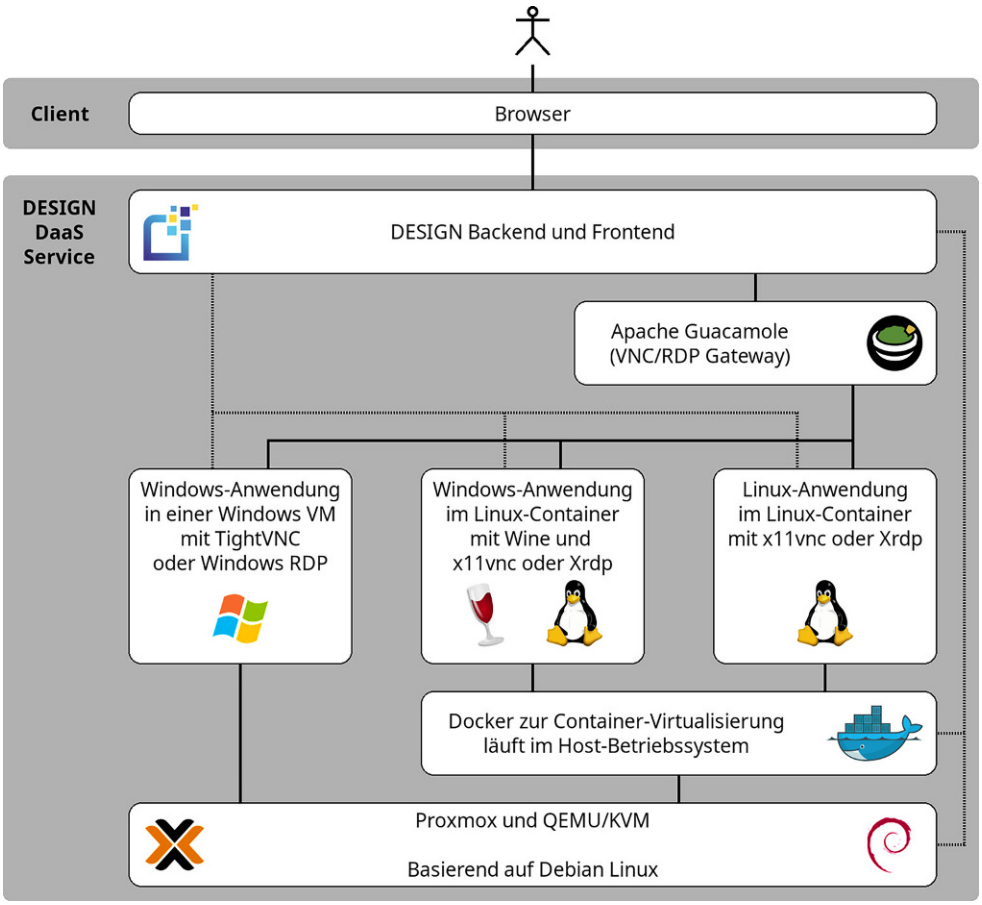


Abb. 1 Architektur und Softwarekomponenten [2]

ten Architektur ermöglichen Linux-Container den Betrieb unveränderter Linux-Anwendungen. Auch Windows-Anwendungen können so betrieben werden, wenn diese mit dem API-Emulator Wine kompatibel sind. Diejenigen Windows-Anwendungen, die nicht mit Wine kompatibel sind, erfordern zum Betrieb virtuelle Maschinen mit dem Betriebssystem Windows. Dieses stellt die einzige proprietäre Softwarekomponente innerhalb der Architektur von DESIGN dar. Alle übrigen Komponenten sind etablierte Open-Source-Lösungen mit gewachsener Stabilität (siehe Anforderung 2).

Zur Fernsteuerung der Linux/Windows-Anwendungen benötigen die Container und Virtuellen Maschinen (VMs) einen entsprechenden Remote-Desktop-Dienst. Für die meisten dieser Protokolle existieren kostenlos nutzbare Server-Implementierungen für Linux und Windows. Die Protokolle VNC und RDP stechen beim Vergleich funktionaler Eigenschaften heraus, da für Sie eine große Anzahl freier Implementierungen aktiv entwickelt und gepflegt werden, die zudem auch in der Lage sind, nur die grafische Benutzeroberfläche einzelner Anwendung zu exportieren und nicht zu jeder Zeit den vollständigen Desktop (siehe Anforderung 3). In Linux-Container integriert DESIGN die freien Server-Implementierungen x11vnc und Xrdp und in Windows-VMs die freie Server-Implementierung TightVNC. Zudem steht in Windows-VMs der bereits standardmäßig im Windows-Betriebssystem enthaltene RDP-Server zur Verfügung.

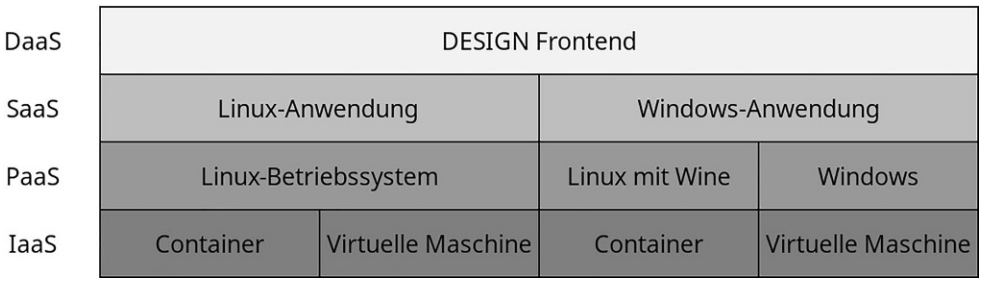


Abb. 2 Integration von Virtualisierungs- und Container-Lösungen

Weitere essenzielle Desktop-Funktionalitäten wie zum Beispiel Drucken, Soundausgabe und Soundaufnahmefunktion sind mit den Protokollen VNC und RDP sowie mit den genannten Server-Implementierungen prinzipiell möglich (siehe Abschnitt Drucken und Sound) und bereits prototypisch in DESIGN implementiert.

Um die Interaktionen komplett via Browser (siehe Anforderung 4) zu ermöglichen, damit die Nutzer keine zusätzliche Client-Software auf Ihren Endgeräten installieren müssen, ist ein Gateway nötig, der die GUI in einen Webstream übersetzt. Zu diesem Zweck integriert DESIGN die Software Apache Guacamole. Diese fungiert als eine Art Broker, der zwischen den VNC- und RDP-Servern und dem lokal installierten Browser vermittelt. Damit kann als Endgerät jedes System genutzt werden, das einen Browser bietet.

Die Open-Source-Servervirtualisierungsplattform Proxmox VE läuft direkt auf den physischen Servern (Bare-Metal) und ermöglicht ohne weitere Softwarekomponenten den Betrieb virtueller Maschinen mit der Kernel-based Virtual Machine (KVM) und containerbasierte Virtualisierung via LXC-Linux-Container. DESIGN verwendet KVM als Hypervisor-Lösung zum Betrieb der Windows-VMs. Die LXC-Funktionalität kommt hingegen nicht zum Einsatz. Gründe dafür sind, dass die LXC-API im Vergleich zur Docker-API vergleichsweise schwach dokumentiert ist und weniger Features bietet. Aus diesen Gründen nutzt DESIGN die Container-Virtualisierungslösung Docker für Linux-Container. Proxmox kann auf lokalen und entfernten Servern laufen (siehe Anforderung 5) und ermöglicht einen Cluster-Betrieb über mehrere physische und virtuelle Server (siehe Anforderung 6). Diese bietet beste Voraussetzungen für ein hohes Niveau an Datenschutz und Datensicherheit und verhindert das Entstehen eines Vendor Lock-in.

Container und virtuelle Maschinen in DESIGN enthalten automatisch einen SSH-Dienst zur nachrichtenorientierten Kontrolle und Steuerung der jeweiligen Container- oder VM-Instanzen und zum Starten und Beenden von Anwendungen innerhalb dieser Container und VMs mithilfe von SSH-Kommandos.

Der SSH-Dienst ermöglicht kryptografisch gesicherte Kommunikation unabhängig von den verwendeten Betriebssystemen innerhalb der Container- und VM-Instanzen.

Durch die zusätzliche Nutzung einer in der Programmiersprache Python selbst entwickelten Softwarekomponente, die DESIGN innerhalb jeder Container- und VM-Instanz automatisch integriert, werden sämtliche erforderlichen Funktionalitäten abgebildet und Plattformunabhängigkeit erreicht. Diese Komponente umfasst generische Methoden zum Anpassen von Bildschirmauflösung, zum Installieren, Starten und Stoppen von Anwendungen sowie zum Abfragen relevanter Statusinformationen wie z. B. IP-Adressen und Hostnamen.

Insbesondere für kleine bis mittlere Nutzerzahlen stellt die Kombination von SSH und selbstentwickelter dedizierter Python-Komponente innerhalb jedes Containers und jeder VM eine praktikable Lösung dar und zeichnet sich durch geringen Konfigurationsaufwand und einen hohen Sicherheitsstandard aus. Nachteilig bei diesem Architekturkonzept ist, dass für jede Nachricht eine neue SSH-Verbindung ausgehandelt wird, die einen signifikanten Overhead erzeugt. Hierbei muss beachtet werden, dass im Kontext eines DaaS zahlreiche kleine Nachrichtenpakete in hoher Frequenz versendet werden. Die Integration einer Message-Oriented Middleware wie zum Beispiel ZeroMQ, RabbitMQ oder Apache Kafka wäre insbesondere bei sehr hohen Nutzerzahlen sinnvoll.

Performance und Latenz

Die Eigenschaft benötigte Daten in angemessener Zeit verarbeiten zu können ist aus unserer Sicht eine der wichtigsten Systemeigenschaften für einen DaaS, da es maßgebliche nichtfunktionale, aber essenzielle Faktoren wie Benutzererfahrung, Reaktionsfähigkeit (englisch: „Responsiveness“) und Gesamteffizienz maßgeblich beeinflusst. In diesem Kontext galt es, im Verlauf der Entwicklung von DESIGN mannigfaltige Herausforderungen zu erkennen und geeignete Lösungswege zu finden. Diese erstrecken sich über mehrere Ebenen der in Abb. 1 gezeigten Architektur und die in den nachfolgenden Abschnitten näher beschrieben sind.

Infrastruktur und Plattform

Die gewählte Infrastruktur, Plattform und verwendete Software haben den größten Einfluss auf den Ressourcenverbrauch und sind daher von größter Bedeutung hinsichtlich der Effizienz des Gesamtsystems. Ein DaaS-System kombiniert alle Aspekte der funktionalen Cloud-Dienstkategorien IaaS, PaaS und SaaS, weshalb insbesondere deren individuelle Leistungscharakteristiken berücksichtigt werden müssen.

Für Infrastructure-as-a-Service (IaaS) bietet unsere Lösung die Flexibilität, entweder virtuelle Maschinen oder Container zu nutzen. Zahlreiche wissenschaftliche Publikationen heben den offensichtlichen Vorteil einer leichtgewichtigen Containerlösung gegenüber virtuellen Maschinen hervor. Dies deutet darauf hin, dass insbesondere im DaaS-Kontext ein containerbasierter Ansatz gegenüber einem virtualisierten Ansatz vorzuziehen ist [5, 9]. Allerdings kann nicht jede Anwendung in einer containerbasierten Umgebung ausgeführt werden, weshalb bei der Entwicklung von

DESIGN sowohl der Betrieb von Anwendungen in Containern als auch in VMs berücksichtigt wurde, um eine möglichst weitreichende Abdeckung potenzieller Anwendungsszenarien zu erzielen.

Für Platform-as-a-Service (PaaS) zeigen Studien, dass es keine dramatischen Leistungsunterschiede gibt, wenn verschiedene Derivate von Betriebssystemen miteinander verglichen werden [1]. Dennoch sind im direkten Vergleich untereinander einzelne Betriebssystemderivate (z. B. Debian, Arch, Fedora, Alpine etc.) performanter und leistungsfähiger [3]. Im Gegensatz dazu zeigen Vergleiche zwischen Betriebssystemen unterschiedlicher Typen (Linux vs. Windows) deutliche Unterschiede, insbesondere bei der Betrachtung der Implementierung von Betriebssystemfunktionalitäten auf niedriger Ebene wie Interrupt Handling, Speicherverwaltung oder Isolierung von Anwendung [10]. Eine signifikante Verbesserung des Ressourcenverbrauchs und der relevanten Leistungsparameter (Bandbreite und Latenz) können in einem hybriden Ansatz erzielt werden, wenn eine Windows-VM durch eine linux-container-basierte Lösung mit integrierter Wine-Laufzeitumgebung ersetzt wird [7].

Im Bereich Software-as-a-Service (SaaS) hängt die Leistung weitgehend von der zugrunde liegenden Infrastruktur bzw. Plattform und der virtualisierten Anwendung selbst ab. Je nach Anwendungsfall können relevante Eigenschaften wie Mehrmandantenfähigkeit (englisch: „Multi-Tenancy“) und verschiedene Bereitstellungsstrategien (Private und Public Cloud) unterschiedlich genutzt werden, um weitere Optimierungspotenziale zu erschließen [12]. Zudem sind auch die genutzten Daten bzw. Datentypen – z. B. Dokumente wie DOCX oder ODT oder Mediendateien wie Filme oder Sounddaten – der konkreten Anwendungen für die Gesamtleistung relevant.

Anwendungsspezifische Optimierungen haben im Einzelfall ein hohes Potenzial zur Verbesserung der Leistung eines DaaS und dessen Ressourcenverbrauch. Bei der Entwicklung eines DaaS wie DESIGN steht aber die möglichst breite Abdeckung potenzieller Anwendungsszenarien im Fokus und nicht die Optimierung des Betriebs einzelner Anwendungen. In diesem Sinne wurden bei der Entwicklung von DESIGN typische Desktop-Anwendungen wie zum Beispiel LibreOffice, Microsoft Word, Notepad++ etc. als geeignete Kandidaten für die Verwendung innerhalb der Lösung in Betracht gezogen und deren korrekte Funktionsfähigkeit war maßgeblich (Abb. 2).

Netzwerklatenz und Bandbreite

Netzwerklatenz und Bandbreite sind maßgebliche Faktoren für die Bereitstellung eines webbasierten Zugriffs auf einen DaaS und dessen enthaltene Anwendungen. Beide Faktoren beeinflussen direkt die vom Nutzer erlebte Benutzererfahrung und Reaktionsfähigkeit und sind somit essenziell für die Benutzbarkeit und Akzeptanz der Nutzer. Die Unterstützung standardisierter Frame-Buffer-Protokolle ermöglicht die generelle Nutzerinteraktion und bietet zusätzlichen Zugriff auf externe Geräte wie Drucker, Soundeingabe und -Ausgabe sowie externe Speichermedien wie zum Beispiel USB-Sticks. Bei der Übertragung von Videoinhalten ist die verfügbare Netzwerkbandbreite häufig der maßgebliche Flaschenhals, insbesondere bei der Übertragung von hochauflösenden Videos oder 3D-Anwendungen. Im Gegensatz dazu ist bei der Übertragung von Audioinhalten, Mausbewegungen oder Tastatureingaben häufig die Netzwerklatenz der relevantere Faktor.

DESIGN integriert eine Reihe von Konfigurationsparametern, um sich an unterschiedliche Anwendungsfälle anzupassen. Zu diesem Zweck wird intern ein guacamole-basiertes Desktop-Gateway verwendet, das weitverbreitete Frame-Buffer-Protokolle wie VNC und RDP integriert. Anwendungsspezifische Anforderungen können durch protokollspezifische Konfigurationsparameter oder die Wahl effizienter Kompressionsalgorithmen bis zu einem gewissen Grad optimiert werden. Zu diesen anwendungsspezifischen Anforderungen gehören die Definition der Art und Weise der Videoübertragung bzw. -kodierung, Art und Weise der Benutzerauthentifizierung, Protokollierung („Logging“), Art und Weise der Druckeranbindung, Einbindung von Audiogeräten, Zwischenablage („Clipboard“), Tastaturlayout etc.

Guacamole integriert bereits Audiounterstützung und ermöglicht die Integration externer Geräte über einen auf der Programmiersprache JavaScript basierenden Web-Client (guacamole.js). Die Proxy-Komponente empfängt HTTP-Anfragen von guacamole.js und wird vom Backend in Form von sogenannten WebSocket Opcodes weitergeleitet. Diese sind im Guacamole-Protokoll definiert [4]. Die Opcodes werden im nächsten Verarbeitungsschritt an den Guacamole-Daemon (guacd) übertragen, der wiederum über die bereits genannten Frame-Buffer-Protokolle VNC oder RDP mit den Containern und VMs kommuniziert (siehe Abb. 3).

Die Integration von Guacamole in das System erfordert jedoch die Berücksichtigung zusätzlicher Faktoren neben den Frame-Buffer-Protokollen VNC und RDP selbst. Authentifizierungsmechanismen sind beispielsweise im DaaS-Kontext unerlässlich, da Benutzer nahtlose Authentifizierung im gesamten System ohne mehrfache Anmeldungen erwarten. Darüber hinaus ist der Schutz sensibler Informationen wie Benutzerpasswörter entscheidend zur Aufrechterhaltung der Systemsicherheit und des Vertrauens der Benutzer. Um das Risiko unbeabsichtigter Manipulationen am DaaS-System zu minimieren, integriert DESIGN die Software Apache Guacamole als Proxy. Dies stellt sicher, dass die Anmeldeinformationen einer generierten Instanz geheim bleiben, während Nutzern dennoch ein autorisier-

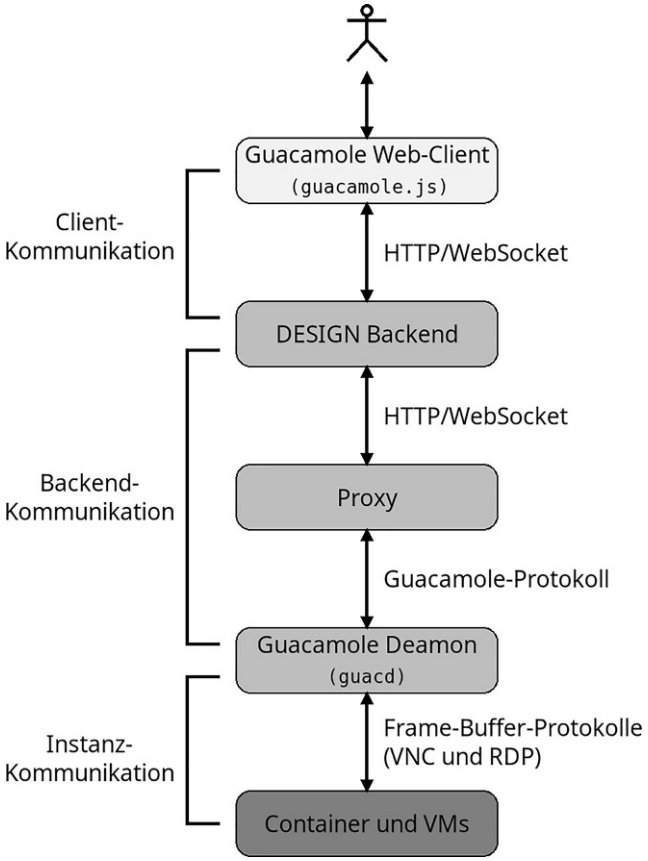


Abb. 3 Kommunikation der Komponenten von DESIGN

ter und passwortloser Zugriff auf die im DaaS betriebenen Anwendungen ermöglicht wird.

Darüber hinaus ermöglicht die Verwendung des Proxys die Unterstützung weiterer zusätzlicher Funktionen wie zum Beispiel die dynamische Anpassung der Bildschirmauflösung. Während Guacamole dies standardmäßig für RDP unterstützt, fehlt eine entsprechende Implementierung für das VNC-Protokoll. Hierbei ermöglicht der Proxy-Ansatz das Abfangen aller ausgetauschten WebSocket Opcodes. Durch das auf diese Art und Weise mögliche Erfassen von Änderungsanforderungen zur Beeinflussung der Bildschirmauflösung können Änderungen der Bildschirmauflösung der Container und VMs umgesetzt werden.

Die Nutzung von Guacamole als Proxy führt allerdings zu einem zusätzlichen Protokoll-Overhead, was abhängig von der Verteilung der genutzten Netzwerkdienste und der zugrunde liegenden Hardware-Infrastruktur zu merklichen Verzögerungen in der Netzwerkkommunikation führt. Trotz dieses zusätzlichen Overheads ermöglicht die beschriebene Architektur einen plattformunabhängigen und nahtlosen Zugriff für alle untersuchten DaaS-Anwendungsszenarien in ausreichender Bandbreite und Latenz.

Interne Verarbeitung und Kommunikation

Ein weiterer kritischer Aspekt, der die Leistungs- und Reaktionsfähigkeit signifikant beeinflusst, betrifft die interne Verarbeitung, Speicherung und Kommunikation der nötigen Daten innerhalb des DaaS. Dies schließt die Erfassung und Verarbeitung von Metadaten sowie den Austausch „echter“ Anwendungsdaten mit allen relevanten Backend-Diensten, Containern und VMs mit ein.

Um eine zufriedenstellende Benutzererfahrung mit einem DaaS zu gewährleisten, müssen bestimmte Komponenten verfügbar sein. Dazu gehören insbesondere der Zugang zu allen infrastrukturellen Backend-Diensten, Datenbanken, Authentifizierungsdienste, Dateisysteme sowie physische und virtuelle Netzwerkgeräte. Zusätzlich ist ein Webserver nötig, um relevante API-Endpunkte für die Frontend-Kommunikation sowie WebSocket-Endpunkte für den Guacamole Viewer bereitzustellen. Generell können all diese Komponenten potenzielle Ressourcenflaschenhälse darstellen. Dies gilt insbesondere bei der Verarbeitung hoher Datenmengen, ausgelöst durch ressourcenintensive Anwendungen oder eine große Anzahl gleichzeitiger Anwendungen (Stichwort: Bandbreite). Ein weiterer potenzieller Auslöser für Engpässe ergibt sich durch die Nutzung von Multimedia- bzw. Echtzeitanwendungen (Stichwort: Netzwerklatenz) im DaaS.

Dies ist besonders dann der Fall, wenn alle Komponenten von derselben Hardware bedient werden. Architektonisch müssen daher alle relevanten Komponenten des DaaS-Systems für die Verteilung auf verschiedene physische Hosts ausgelegt sein, damit das Gesamtsystem im Falle von Engpässen horizontal skalieren kann. Dennoch kann es erforderlich sein, bestimmte Kernkomponenten direkt auf den Host-Systemen („Bare-Metal“) zu betreiben, um Zugriffe mit möglichst geringer Latenz zu gewährleisten.

Erfahrungen während der Entwicklung von DESIGN zeigen, dass Komponenten, die niedrige Latenzzeiten und damit einen echtzeitnahen Betrieb gewährleisten, physisch so nah wie möglich am Host-System platziert werden müssen (siehe Abb. 4). Dies umfasst alle Dienste zur Datenverarbeitung und Kommunikation, die zur Übertragung von Audio, Video, Maus und Tastatur relevant sind, sowie alle infrastrukturellen Backend-Dienste zum Betrieb von Containern und VMs. Konkret handelt es sich bei diesen Komponenten um die Backend-Dienste Proxmox (mit der Virtualisierungslösung QEMU/KVM für virtuelle Maschinen), die Container-Virtualisierungslösung Docker und das Apache Guacamole Proxy.

Falls aufgrund beschränkter Hardware-Eigenschaften oder sehr hoher Nutzerzahlen der Betrieb der genannten Komponenten direkt auf dem jeweiligen Host-System möglich ist, sollten die entsprechend verteilten Dienste zumindest mit demselben physischen Netzwerk verbunden

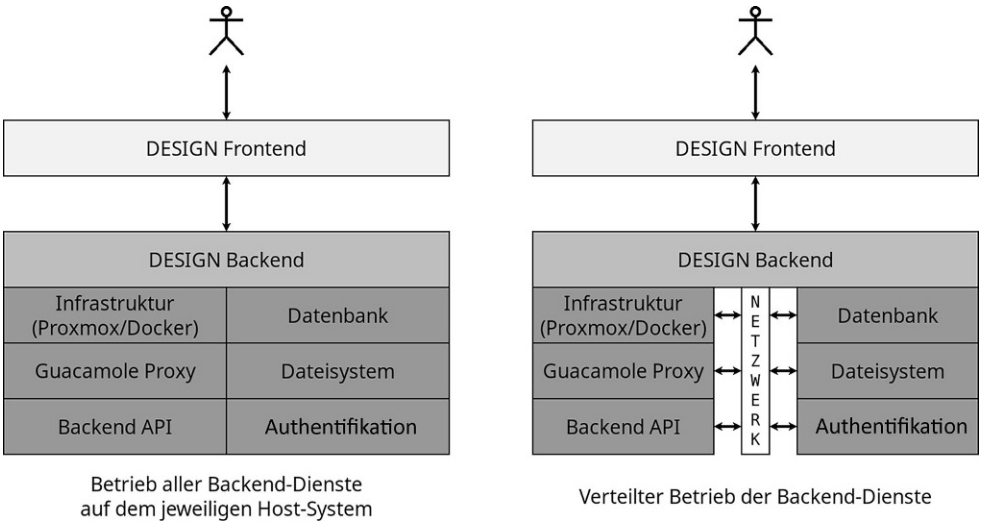


Abb. 4 Interner Kommunikationsfluss

sein, um Netzwerklatenzen bestmöglich zu minimieren. Alle anderen Komponenten (z. B. die Datenbank mit den Metadaten zu den Containern, VMs, Anwendungen und Verbindungen, Das CEPH-Dateisystem, der Authentifizierungsdienst oder der selbst entwickelte Monitoring-Dienst) können ohne größere Leistungseinbußen auf dedizierte Systeme verteilt werden (siehe Abb. 4).

Obwohl zum aktuellen Stand der Entwicklung noch keine vollständige Leistungsanalyse vorliegt, konnten in einer lokalen Testumgebung bereits vielversprechende Ergebnisse erzielt werden. Je nach Konfiguration bietet DESIGN akzeptable Rundlaufzeiten (englisch: „Round-Trip Time“ bzw. RTT) und Verzögerungen (Latenzen) für alle relevanten Kommunikationswege (siehe Abb. 3).

In einer Minimalkonfiguration mit einem physischen Server wurden Verzögerungen von etwa 10 ms für die Client-Kommunikation, etwa 100 ms für die interne Backend-Kommunikation und bis zu 1000 ms für die Instanzkommunikation gemessen (siehe Abb. 3). Geplante, aber noch nicht vollständig implementierte Optimierungskonzepte wie zum Beispiel die Parallelisierung von Hintergrundaufgaben können in Zukunft die Performance deutlich verbessern.

Drucken und Sound

Neben den Hauptanforderungen des Systems existieren zusätzliche Funktionalitäten, die aus Nutzersicht häufig essenziell sind. Dazu zählen insbesondere die Einbindung von Audiogeräten zur Ein- und Ausgabe sowie die Einbindung von Druckern.

Der in Windows-Betriebssystemen integrierte RDP-Dienst unterstützt die Audioübertragung. Guacamole unterstützt ebenfalls die Weiterleitung an den Client. Die Audioausgabe ist standardmäßig aktiviert und wird automatisch im Browser abgespielt. Um die Audioeingabe (Mikrofon) zu aktivieren, wird beim Verbindungsaufbau ein entsprechender Parameter gesetzt. Die Guacamole-Frontend-Bibliothek enthält eine Implementierung für einen Audiorecorder, der die Audioaufnahme des Browsers an den Server überträgt. Den Startzeitpunkt und das Ende einer Audioaufnahme steuert das RDP-Protokoll selbständig und präzise. Der in der Architektur (siehe Abb. 1) von DESIGN für Linux-Container vorgesehene Xrdp-Server unterstützt die Audioübertragung ebenso.

VNC unterstützt grundsätzlich keine Audioübertragung. Allerdings besteht hier die Möglichkeit, die Audioausgabe über einen PulseAudio-Server, wie er standardmäßig in den meisten Linux-Distributionen enthalten ist, direkt an Guacamole weiterzuleiten. Apache Guacamole kombiniert die VNC-Daten mit den Audiodaten aus PulseAudio und leitet sie durch das Guacamole-Protokoll an den Client weiter. Eine Audioeingabe (Mikrofon) ermöglicht weder das VNC-Protokoll noch der Guacamole-Dienst. Als Hilfskonstruktion, um Audioeingabe dennoch zu ermöglichen, integriert DESIGN einen zusätzlichen Audioserver auf den Linux Containern, der die vom Client (Browser) des Benutzers aufgenommenen Audiodaten an das lokale virtuelle Mikrofon der jeweiligen Linux-Instanz weiterleitet. Vom Browser (Client des Benutzers) zum Backend von DESIGN werden Audioübertragungen über ein WebSocket gesendet. Vom Backend zu den jeweiligen Linux-Containern erfolgt die Übertragung via TCP-Protokoll. Als Server auf den Containern kann beispielsweise Netcat zum Empfang und Forwarding der Audioübertragungen an das lokale virtuelle Mikrofon verwendet werden. Der Client muss dafür eine weitere Verbindung aufbauen, da die Übertragung über das Guacamole-Protokoll nicht möglich ist. Ein Nachteil dieses Konzepts ist, dass zwei Verbindungen zum gleichen Zweck gleichzeitig existieren müssen. Konkret handelt es sich dabei um die Verbindung zwischen Client und Guacamole via

Guacamole-Protokoll zur Übertragung der Bildinformationen und der Audioausgabe sowie die Verbindung zwischen Client und dem zusätzlichen Audioserver zur Audioeingabe. Daraus resultiert, dass die Audioeingabe eventuell verzögert übertragen und asynchron wahrgenommen wird.

Die Entwicklung und Implementierung einer Druckfunktionalität ist auf verschiedene Art und Weise möglich. Eine Möglichkeit ist die Freigabe des lokalen Druckers im lokalen Netzwerk. Somit können Druckaufträge über das Internet Printing Protocol (IPP) oder eine vergleichbare Lösung an den Drucker geschickt werden. Ein Nachteil dieses Konzepts ist, dass Konfigurationsarbeiten auf Seite des Nutzers nötig sind, was einem der Kernziele von DESIGN widerspricht.

Eine alternative und weniger aufwendige Lösung ist, zu druckende Dokumente als PDF-Dateien über den Browser herunterzuladen und danach ganz normal auszudrucken. Diese Funktionalität enthält Guacamole bereits standardmäßig für das RDP-Protokoll. Konkret stellt Guacamole einen virtuellen Drucker für die jeweilige RDP-Session zur Verfügung. Dieser kann durch Linux-Container und Windows-VMs genutzt werden, um PDFs zu erzeugen und diese an das DESIGN-Backend zu senden. Diese werden dann über die Backend-Komponente von DESIGN an den Client (Browser) übertragen, der das fertige PDF herunterladen und an einem lokal verfügbaren Drucker ausdrucken kann.

Das Protokoll VNC bietet keine vergleichbare Option. Eine gangbare Lösung ist der Betrieb eines eigenen, dedizierten Druckerservers, der virtuelle Drucker zur Erzeugung von PDF-Dateien realisiert. Somit können Container und VMs eigene Session-Drucker zugewiesen bekommen. Die erzeugten PDF-Dateien werden über eine WebSocket-Verbindung an den Client (Browser) geschickt, der sie herunterladen und lokal ausdrucken kann. Es besteht auch die Möglichkeit, die erzeugten PDF-Dateien über die Backend-Komponente von DESIGN an einen lokal verfügbaren und zuvor konfigurierten Drucker zu übertragen.

Somit kann zusammenfassend gesagt werden, dass für die Integration Drucken und Sound beim Protokoll RDP keine Änderungen an der Architektur (siehe Abb. 1) nötig sind, da beide Funktionalitäten über das Guacamole-Protokoll bereits implementiert sind. Für das Protokoll VNC

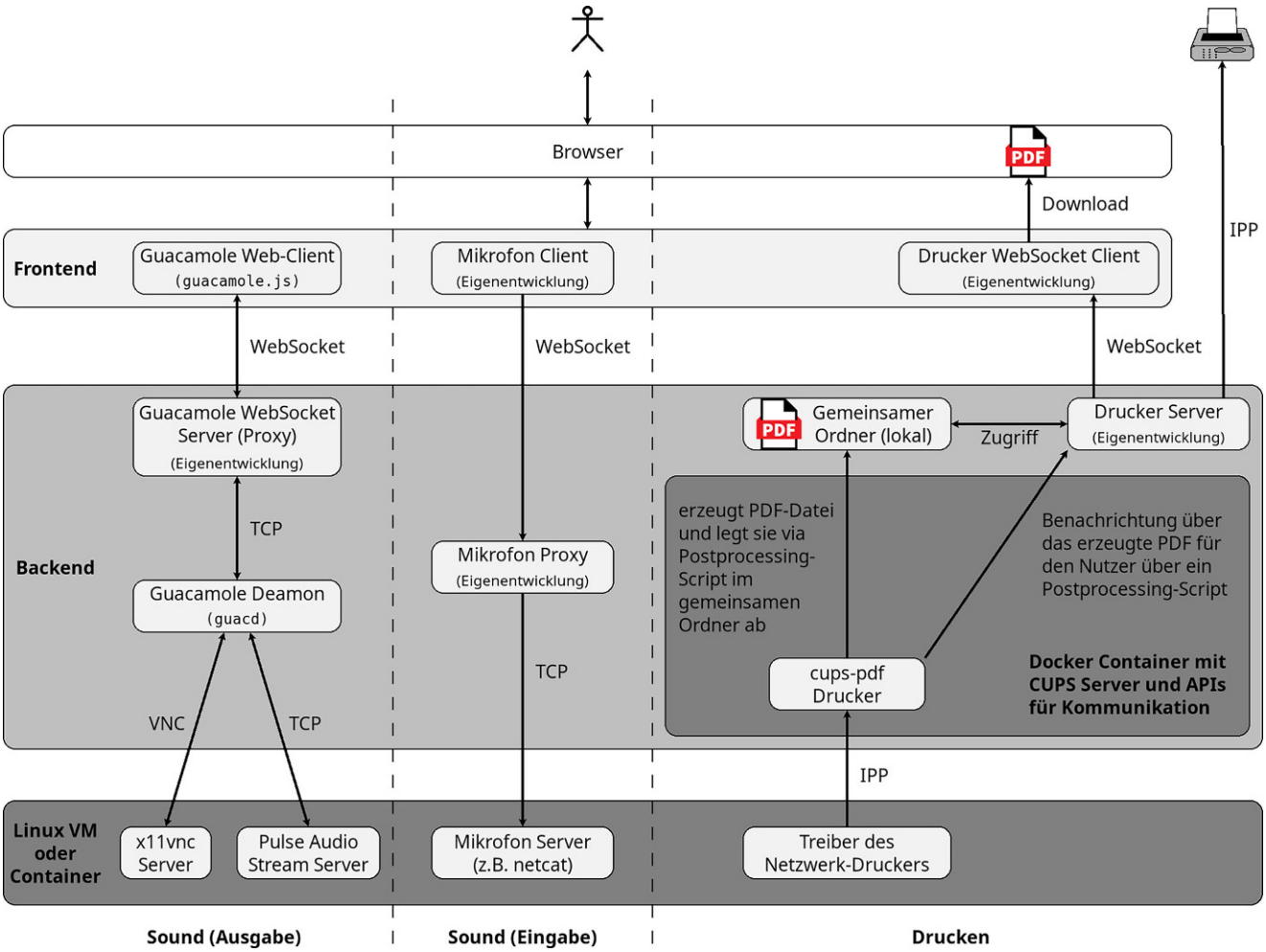


Abb. 5 Drucken und Sound in DESIGN via das Protokoll VNC

Tab. 3 Vergleich von Drucken und Sound in den Protokollen für RDP und VNC

Funktionalität	RDP	VNC
Audioausgabe	Nativ über das Guacamole-Protokoll	Über einen PulseAudio-Server
Audioeingabe	Nativ über das Guacamole-Protokoll	Über einen zusätzlichen Server zum Empfang und zur Weiterleitung der Audioinformationen
PDF-Drucker	Nativ über das Guacamole-Protokoll	Über einen dedizierten Druckerserver

müssen die entsprechenden Audio- und Druckerkomponenten extra hinzugefügt werden (siehe Abb. 5). Tab. 3 zeigt den Grad der Unterstützung von Drucken und Sound in den Protokollen RDP und VNC sowie bereitgestellte Erweiterungen.

Zukünftige Erweiterungen

Die Entwicklung und Implementierung von DESIGN war herausfordernd und spannend, da zahlreiche etablierte Technologien und Protokolle sowie selbst entwickelte Teillösungen zu einem größeren Ganzen kombiniert wurden. Das selbstgesteckte Ziel einer flexiblen, stabilen, benutzerfreundlichen und neuartigen Desktop-as-a-Service-Lösung, die die Grenzen bekannter Bereitstellungsstrategien auflöst, wurde größtenteils erreicht.

Noch anstehende Aufgaben für die nahe Zukunft sind die Verbesserung der Benutzbarkeit u. a. durch ein Template-System für neue Anwendungen sowie die Verbesserung der Gesamtperformance durch die Parallelisierung von Hintergrundaufgaben. Grundlage hierfür ist auch eine eingehende Performanceanalyse, deren Durchführung und Auswertung noch aussteht.

Mittelfristig geplant ist weiterhin die Integration einer Message-Oriented Middleware wie zum Beispiel ZeroMQ, RabbitMQ oder Apache Kafka. Eine solche Middleware ist insbesondere bei sehr hohen Nutzerzahlen sinnvoll, da sie die auftretende Last besser auf den vorhandenen Hardware-Ressourcen verteilen kann und damit die definierten Performanceziele von DESIGN in besonderem Maße unterstützen würde.

Auf lange Sicht sollte der Fokus des Projekts auf der Erweiterung der Cluster-Fähigkeit sowie der Optimierung der bis jetzt implementierten Komponenten liegen. In Bezug auf virtuelle Maschinen konnten im bisherigen Projektverlauf alle erforderlichen Voraussetzungen für einen skalierbaren Cluster-Betrieb erreicht werden. Für die im System erzeugten Container-Instanzen fehlt ein äquivalenter Ansatz bisher. Diesem Umstand kann perspektivisch durch die Anbindung etablierter Cluster-Lösungen für Container (wie z. B. Kubernetes) begegnet werden, um somit sämtliche Hosting-Strategien auch im Cluster-Betrieb zu ermöglichen.

Fazit

DESIGN ist eine vielversprechende Alternative zu bestehenden Desktop-as-a-Service-Lösungen, indem es die Vorteile von SaaS und nativen Anwendungen kombiniert und deren Nachteile minimiert. Der browserbasierte Ansatz ermöglicht eine einfache Integration und Nutzung von Anwendungen ohne zusätzlichen Verwaltungsaufwand.

Förderung Die Entwicklung und prototypische Implementierung der in diesem Artikel beschriebenen neuartigen DaaS-Lösung erfolgte im Rahmen des Projekts DESIGN, das vom Bundesministerium für Wirtschaft und Klimaschutz als gefördertes ZIM-Kooperationsprojekt finanziert wurde. Alle beschrieben Konzepte und Lösungen wurden gemeinsam mit dem Projektpartner NUROMEDIA GmbH entwickelt.

Funding Open Access funding enabled and organized by Projekt DEAL.

Interessenkonflikt A. Atanassov, C. Baun und J. Bouché geben an, dass kein Interessenkonflikt besteht.

Open Access Dieser Artikel wird unter der Creative Commons Namensnennung 4.0 International Lizenz veröffentlicht, welche die Nutzung, Vervielfältigung, Bearbeitung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden.

Die in diesem Artikel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist für die oben aufgeführten Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.

Weitere Details zur Lizenz entnehmen Sie bitte der Lizenzinformation auf http://creativecommons.org/licenses/by/4.0/deed.de.

Literatur

1. Balen J, Vdovjak K, Martinović G (2020) Performance evaluation of windows virtual machines on a linux host. Autom Časopis Za Autom Mjerenje Elektroniku Računarstvo I Komunikacije 61(3):425–435
2. Baun C, Bouche J (2023) Closing the gap between web applications and desktop applications by designing a novel desktop-as-a-service (daas) with seamless support for desktop applications. Open J Cloud Comput 8(1):1–19
3. Boras M, Balen J, Vdovjak K (2020) Performance evaluation of linux operating systems. In: Int. Conf. on Smart Systems and Technologies. IEEE, S 115–120

4. Guacamole protocol reference. https://guacamole.apache.org/doc/gug/protocol-reference.html
5. Felter W, Ferreira A, Rajamony R, Rubio J (2015) An updated performance comparison of virtual machines and linux containers. In: Int. symposium on performance analysis of systems and software (ISPASS). IEEE, S 171–172
6. Garmpis A, Gouvatsos N (2016) Design and development of webubu: an innovating web-based instruction tool for linux os courses. Comput Appl Eng Educ 24(2):313–319
7. Huang C, Chen J, Zhang L, Luo Q (2012) Performance evaluation of virtualization technologies for windows programs running on linux operating system. In: Int. Conf. on Network Computing and Information Security. Springer, S 759–766
8. Liu K, Dong L (2012) Research on cloud data storage technology and its architecture implementation. Procedia Eng 29:133–137
9. Potdar AM, Narayan D, Kengond S, Mulla MM (2020) Performance evaluation of docker container and virtual machine. Procedia Comput Sci 171:1419–1428
10. Sulaiman NS, Raffi ASHA (2021) Comparison of operating system performance between windows 10 and linux mint. Int J Synerg Eng Technol 2(1):92–102
11. VidyaBanu R, Preethi J, Dinesh N (2011) Implementation of financial system using EyeOS in the cloud environment. In: Int. Conf. on Recent Trends in Information Technology. IEEE, S 656–660
12. Zhong L, Wo T, Li J, Li B (2010) A virtualization-based saas enabling architecture for cloud computing. In: 6th Int. Conf. on Autonomic and Autonomous Systems. IEEE, S 144–149

Hinweis des Verlags Der Verlag bleibt in Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutsadressen neutral.